



UNIVERSIDAD
DEL QUINDÍO®

Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

Documento Técnico - Seguimiento 1

Miguel Angel Marin Montes
Johan Camilo Garcia Garcia



UNIQUINDÍO
en conexión territorial

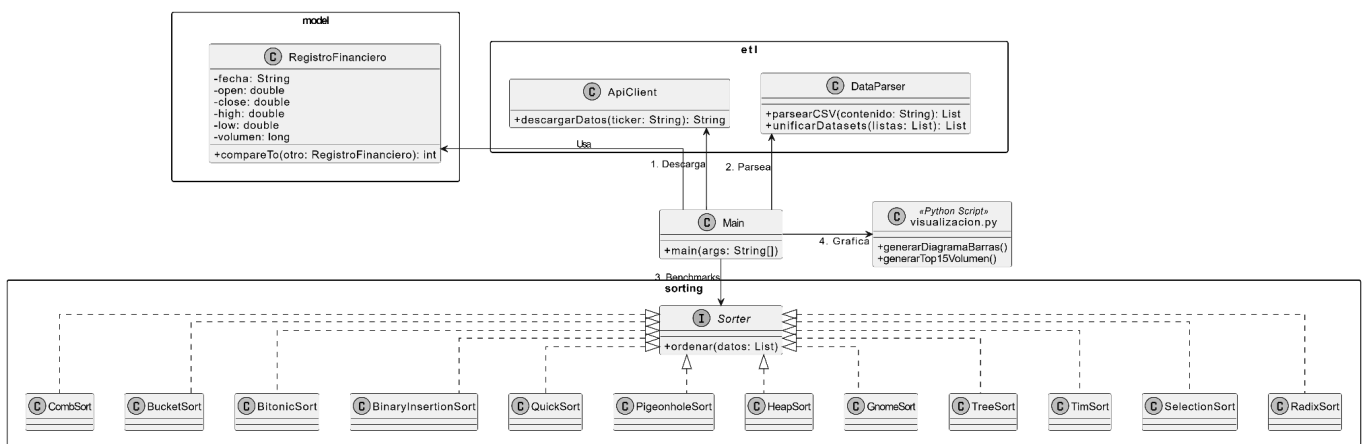
www.uniquindio.edu.co

1. Introducción

- 1.1. Contexto del Proyecto:** El presente proyecto se centra en el diseño e implementación de un sistema algorítmico capaz de procesar y ordenar masivamente históricos de datos financieros. El reto principal consiste en estructurar colecciones de objetos aplicando una lógica de ordenamiento propuesto por los criterios: el orden principal es estrictamente cronológico (basado en la Fecha), utilizando el Precio de Cierre (Close) como criterio secundario de comparación. El sistema hace uso de herramientas de ordenamiento nativas de Java, exigiendo la construcción manual desde cero de algunas estructuras de datos y enfoques algorítmicos, teniendo como soporte visual el uso de gráficos de barra, con la finalidad de presenciar las grandes diferencias entre los tiempos de ejecución de algunos algoritmos de ordenamiento
- 1.2. Objetivos:** El objetivo principal es clasificar correctamente el conjunto de datos financieros mediante la implementación de doce algoritmos de ordenamiento clásicos. Asimismo, se busca establecer un laboratorio de pruebas que permita contrastar el rendimiento de estos mismos mediante el uso correcto de las gráficas, la estabilidad y la complejidad temporal de cada algoritmo frente a un volumen real de información.

2. Arquitectura y Diseño del Sistema

- 2.1. Diagrama de Clases (UML):**



- 2.2. Estructuras de Datos Propias:** Para cumplir con las restricciones del proyecto y evitar el uso de colecciones nativas de Java, se diseñó e implementó una estructura de datos personalizada basada en Nodos y Colas Enlazadas Simples. Esta estructura fue vital para el funcionamiento del algoritmo Radix Sort. Cada Nodo almacena una referencia a un objeto de tipo RegistroFinanciero y un puntero al siguiente nodo. La clase ColaEnlazada gestiona los punteros head (frente) y tail (final) para permitir operaciones de encolado (enqueue) y desencolado (dequeue).
- 2.3. Transformación de Datos para Evaluación:** Para optimizar las comparaciones y habilitar algoritmos no comparativos (como Radix o Bucket Sort), se implementó una técnica de transformación de datos (Data Parsing):

Fechas: Las fechas originales (ej: 2023-05-12) se transformaron a un formato numérico entero AAAAMMDD (ej. 20230512). Esto permite que la comparación cronológica se reduzca a una evaluación matemática de enteros, siendo mucho más rápida que comparar objetos de tipo String o Date.

Precios: Los precios de cierre se multiplicaron por 100 y se castearon a enteros (transformando dólares a centavos). Esto elimina los problemas de precisión de los tipos de datos double o float.

● 2.4. Visualización de Datos: Integración con Python

Java es un lenguaje muy robusto para manejar la lógica de bajo nivel, la gestión de memoria y la ejecución de los algoritmos de ordenamiento, pero construir gráficos estadísticos directamente en java es un proceso rígido. Por esta razón, se tomó la decisión usar Python, un lenguaje que destaca por su agilidad y su vasto ecosistema orientado al análisis de datos y programación en el ámbito financiero.

Para lograr este objetivo, se construyó un script independiente (visualizacion.py) que actúa como el motor gráfico del proyecto, apoyándose en dos librerías fundamentales:

Pandas (El lector de datos): En lugar de enviar los datos uno por uno entre los dos lenguajes, Java guarda los resultados finales (los tiempos de ejecución y el Top 15 de volumen) en archivos .csv. Pandas es la librería encargada de leer estos archivos de texto de forma casi instantánea y convertirlos en "DataFrames" (estructuras similares a tablas dinámicas), facilitando su manipulación.

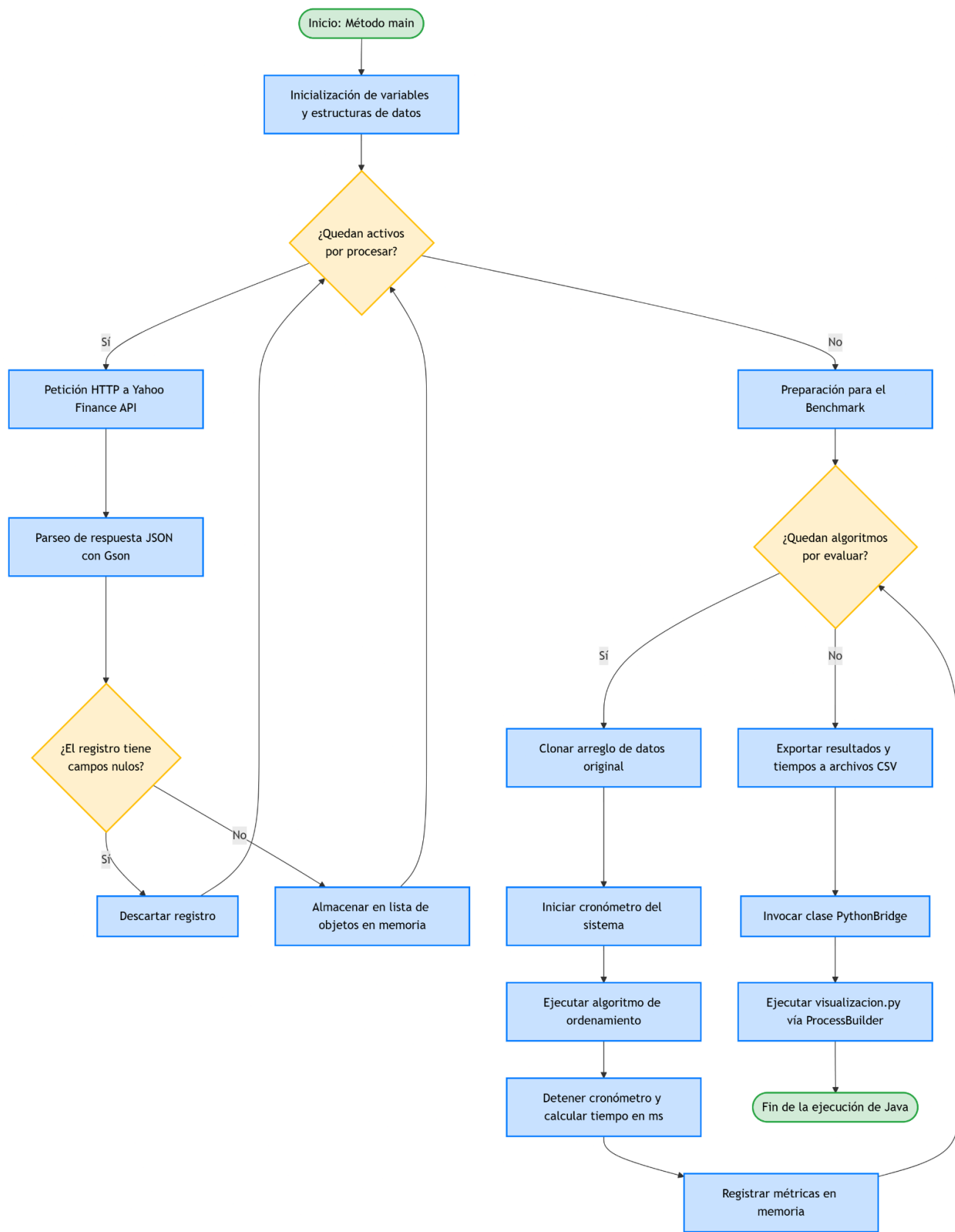
Matplotlib / Seaborn: Una vez que los datos están estructurados, estas librerías toman el control para dibujar los diagramas de barras. Se encargan de calcular automáticamente la escala de los ejes (milisegundos para el tiempo, cantidades para el volumen), aplicar paletas de colores legibles y generar las etiquetas correspondientes para cada algoritmo o fecha.

Automatización del Proceso (El Puente): Para cumplir con el requerimiento de reproducibilidad del proyecto, era vital que el usuario no tuviera que ejecutar procesos manuales por separado. Para ello, se implementó una clase puente en Java (PythonBridge) que utiliza la herramienta ProcessBuilder. Gracias a esto, en el instante exacto en que Java termina de ordenar y guardar los .csv, le da una "orden" interna al sistema operativo para que ejecute el script de Python en segundo plano. Así, el usuario obtiene las gráficas generadas y guardadas como imágenes de forma completamente automática y transparente.

● 2.5. Diagrama de flujo principal generalizado:



Documento Tecnico



3. Análisis e Implementación de Algoritmos

- **3.1. Radix Sort (Distribución por Colas Enlazadas):** Algoritmo de distribución no comparativo. Para cumplir con las restricciones de no usar colecciones nativas de Java, se construyeron estructuras de datos propias (NodoRegistro y ColaRegistros) simulando un comportamiento FIFO. Para resolver el criterio compuesto (Fecha y Precio).

Paso a Paso:

1. Se inicializa un arregloCubetas que contiene 10 instancias de ColaRegistros, representando los dígitos del 0 al 9.
2. Se invoca el método ejecutarFaseOrdenamiento dos veces: primero ordenando por Precio de Cierre (evaluarFecha = false) y luego por Fecha (evaluarFecha = true). Esto garantiza evaluar si son fechas iguales.
3. El algoritmo busca linealmente el valor máximo y calcula la cantidad de posiciones decimales iterables (totalDigitosRequeridos) mediante la longitud matemática del número (String.valueOf().length()).
4. Se itera sobre las posiciones decimales. Por cada registro, se transforma su valor a número entero y se invoca extraerDigitoEnPosicion (usando Math.pow y módulo % 10) para aislar el dígito. El registro se encola en la cubeta correspondiente.
5. Se recorren las 10 cubetas en orden. Mientras no carezcan de elementos (!careceDeElementos()), se extraen los registros (extraerRegistro()) sobrescribiendo la lista original mediante el puntero indiceReasignacion.

- **3.2. Binary Insertion Sort:** En lugar de realizar una búsqueda lineal hacia atrás iteración por iteración para encontrar dónde insertar un elemento, aprovecha que la parte izquierda del arreglo ya está ordenada para realizar una búsqueda binaria, reduciendo drásticamente las comparaciones.

Paso a Paso:

1. El algoritmo itera desde el indiceActual = 1 hasta el final de la lista. En cada ciclo, aísla el RegistroFinanciero a evaluar.
2. Se llama al método busquedaBinaria, definiendo los límites desde la posición 0 hasta el indiceActual - 1 (la sublista ya ordenada).
3. Dentro de la búsqueda binaria, calcula el centro. Si el elemento central es menor al dato evaluado (usando el método esEstrictamenteMenor que transforma fechas a YYYYMMDD), el límite inferior avanza. Crucialmente, para mantener la *estabilidad* del ordenamiento en fechas exactas, si los valores son iguales, el algoritmo avanza hacia la derecha (limInf = centro + 1).



4. Cuando los límites se cruzan ($\text{limInf} > \text{limSup}$), retorna la posición exacta de inserción.
5. Finalmente, realiza un desplazamiento de los elementos mayores hacia la derecha mediante un bucle while e inserta el dato en el hueco generado.

- **3.3. Quick Sort: Descripción General:** Algoritmo recursivo basado en "Divide y Vencerás". Utiliza un elemento llamado *pivote* para particionar el arreglo, asegurando que todos los elementos menores queden a la izquierda y los mayores a la derecha.

Paso a Paso:

1. Se define el rango inicial a ordenar ($\text{inferior} = 0$ y $\text{superior} = \text{tamaño} - 1$).
2. Se selecciona el pivote ubicándose en la mitad exacta del subarreglo actual: $(\text{inferior} + \text{superior}) / 2$.
3. Se utilizan dos punteros (i por la izquierda, j por la derecha). El puntero i avanza ignorando registros menores al pivote, y el puntero j retrocede ignorando registros mayores.
4. Cuando ambos punteros se detienen, significa que encontraron elementos en el lado equivocado. Si los punteros no se han cruzado ($i \leq j$), se intercambian esos dos RegistroFinanciero y los punteros avanzan.
5. Este ciclo se repite hasta que i y j se cruzan. Finalmente, se realizan dos llamadas recursivas independientes: una para la partición izquierda y otra para la derecha.

- **3.4. Heap Sort:** Este algoritmo organiza la colección como si fuera un Árbol Binario implícito (Max-Heap) dentro del mismo arreglo. Su objetivo es asegurar que cada nodo padre sea estrictamente mayor que sus hijos, para luego extraer la raíz progresivamente.

Paso a Paso:

1. Itera desde la mitad de la lista hacia atrás ($n / 2 - 1$), llamando al método `construirMonticulo`. Esto organiza todo el arreglo inicial como un Max-Heap válido.
2. Dentro de `construirMonticulo`, calcula las posiciones virtuales de los hijos ($2*i + 1$ y $2*i + 2$). Usa `esEstrictamenteMenor` para verificar si algún hijo es mayor que su padre. Si lo es, intercambia el nodo padre por el hijo mayor y hace una llamada recursiva hacia abajo para reparar el subárbol afectado.
3. Una vez armado el Max-Heap, el registro mayor de toda la colección se encuentra en la posición 0.



4. Intercambia el elemento de la posición 0 con el de la última posición disponible i. El elemento mayor ya queda definitivamente ordenado al final.
5. Llama de nuevo a construirMonticulo solo para la posición 0 para que el segundo registro mayor "suba" a la raíz, repitiendo el ciclo.

- **3.5. Gnome Sort:** Un algoritmo de intercambio que simula el comportamiento de un gnomo de jardín (como su nombre lo dice) que organiza macetas. Avanza de forma lineal comparando pares adyacentes; si encuentra un desorden, intercambia y da pasos hacia atrás hasta ubicar el elemento en su sitio correcto.

Paso a Paso:

1. Inicia con `indiceActual = 0`. Si se encuentra en 0, avanza al índice 1 automáticamente por falta de un elemento previo para comparar.
2. Compara el registro en `indiceActual` con el de la posición anterior (`indiceActual - 1`) mediante `esEstrictamenteMenor`.
3. Si el registro actual es mayor o igual (están en orden), el algoritmo simplemente da un paso adelante (`indiceActual++`).
4. Si el registro actual es estrictamente menor (están desordenados), intercambia ambos registros y obliga al puntero a retroceder (`indiceActual--`). Esto desencadena una serie de retrocesos en cascada hasta que el registro intercambiado "cae" en su posición correcta.

- **3.6. Bitonic Sort:** Algoritmo de comparación paralelo que construye secuencias "bitónicas" (mitad ascendente, mitad descendente) y luego las fusiona. Su estructura matemática nativa exige que la cantidad de datos sea una potencia de 2.

Paso a Paso:

1. Llama al método recursivo `ejecutarBitonicSort` únicamente sobre esta porción de datos válida.
2. Este método divide la porción en dos mitades. Ordena la primera mitad de forma ascendente (`true`) y la segunda de forma descendente (`false`).
3. Llama a `fusionarBitonic`, el cual realiza comparaciones con un salto o brecha de tamaño mitad utilizando el método `compararEIntercambiar`.
4. Finalmente, como quedaron elementos fuera de la potencia de 2, invoca el método `ordenamientoResidual` (un algoritmo de inserción) para ubicar esos elementos finales en la lista ya semi-ordenada.



- **3.7. Tim Sort:** Algoritmo híbrido altamente optimizado que aprovecha el rendimiento de Insertion Sort para listas pequeñas y el poder de Merge Sort para la integración de grandes volúmenes de datos.

Paso a Paso:

1. El algoritmo inicia dividiendo la colección total en pequeños bloques lógicos (Runs) de tamaño $MIN_MERGE = 32$.
2. Aplica un método insertionSort adaptado para ordenar internamente cada uno de estos bloques, pasando los límites left y right para no afectar el resto del arreglo.
3. Una vez que todos los bloques de tamaño 32 están ordenados, comienza la fase iterativa de fusión (Merge). El tamaño del bloque a fusionar se duplica en cada pasada ($size = 2 * size$).
4. El método merge calcula los puntos de corte (left, mid, right) y extrae los datos a dos listas temporales (leftList y rightList).
5. Utilizando el método .compareTo(), compara frente a frente los elementos de ambas listas temporales, sobrescribiendo la lista original en orden ascendente y finalmente anexando los elementos sobrantes de cualquiera de las dos mitades.

- **3.8. Comb Sort:** Inicia comparando registros muy distantes. Este "gap" (brecha, es un espacio vacío en un gráfico de precios. Ocurre cuando el precio de un activo abre más alto o más bajo que su cierre anterior) se reduce iterativamente, eliminando velozmente los registros pequeños atrapados al final de la colección (conocidos como "tortugas").

Paso a Paso:

1. Inicia definiendo el gap igual al tamaño total de la colección.
2. Dentro de un ciclo while, calcula el nuevo gap dividiendo el anterior por un factor de contracción ideal de 1.3 (implementado de forma eficiente sin coma flotante como $(gap * 10) / 13$).
3. Se recorre la lista comparando el elementoActual en la posición i y el elementoSiguiente en la posición i + gap valiéndose del método .compareTo().
4. Si están invertidos, ejecuta un intercambio clásico con una variable temp de tipo genérico T.
5. Activa la bandera swapped = true. El ciclo principal se detiene solo cuando el gap es estrictamente 1 y ya no se registran más intercambios.

- **3.9. Selection Sort:** Algoritmo comparativo de fuerza bruta. Divide lógicamente el arreglo en una sublista ordenada y otra desordenada, buscando exhaustivamente el elemento mínimo en cada pasada para colocarlo al final de la sección ordenada.

Paso a Paso:

1. Se utiliza un bucle externo i que va de 0 hasta $n - 1$, representando el límite actual de los datos ordenados. Se asume temporalmente que este índice es el menor ($\text{minIdx} = i$).
2. Se inicia un bucle interno j desde la posición $i + 1$ hasta el final de la colección.
3. Se invoca el método `.compareTo()` propio de la interfaz genérica. Si el elemento en j es estrictamente menor que el elemento en minIdx (retorna un valor < 0), se actualiza el puntero $\text{minIdx} = j$.
4. Al finalizar la búsqueda exhaustiva del bucle interno, se verifica si el minIdx cambió respecto al inicio ($\text{minIdx} \neq i$).
5. Si hubo cambio, se ejecuta el intercambio apoyándose en una variable `temp` de tipo `RegistroFinanciero`, moviendo el mínimo absoluto encontrado a su posición definitiva.

- **3.10. Tree Sort:** Algoritmo basado en la construcción de un Árbol Binario de Búsqueda (BST). Los datos se insertan secuencialmente ramificándose a la izquierda (menores) o derecha (mayores), para luego extraerse ordenados mediante un recorrido *In-Order*.

Paso a Paso:

1. Se define una clase interna privada `Node`, la cual actúa como contenedor genérico del valor y guarda los punteros a sus hijos `left` y `right`.
2. Se limpia la raíz ($\text{root} = \text{null}$) y se recorre la lista original iterativamente, pasando cada elemento al método `insertarRec`.
3. Durante la inserción, el método `.compareTo()` define el rumbo. Si el valor es menor a la raíz actual, el flujo se va recursivamente a `root.left`. Crucialmente, si el valor es mayor o igual, se va a `root.right`; esta lógica gestiona los duplicados y mantiene la estabilidad del criterio compuesto.
4. Una vez construido el árbol, se inicializa un arreglo de un solo elemento `int[] index = {0}` para llevar el conteo posicional por referencia.
5. Se ejecuta el método `inOrderRec`. Este visita toda la rama izquierda (menores), luego guarda el valor de la raíz actual sobrescribiendo la lista original (`lista.set(index[0], root.value)`), incrementa el índice, y finalmente visita la rama derecha (mayores), logrando un ordenamiento perfecto.

- **3.11. Pigeonhole Sort:** Algoritmo no basado en comparaciones directas (Distribución). Funciona creando un "casillero" (pigeonhole) para cada posible valor dentro del rango existente en la lista, agrupando los elementos y luego recolectándolos secuencialmente.

Paso a Paso:

1. Utiliza una interfaz funcional keyExtractor (convierte el registro a entero numérico) para escanear toda la lista e identificar el valor mínimo y máximo.
 2. Calcula el rango matemático de cajas necesarias restando $\text{max} - \text{min} + 1$.
 3. Crea una lista de Listas (pigeonholes) con capacidad equivalente al rango. Es obligatorio que sean listas para manejar colisiones de elementos duplicados (ej. varias cotizaciones en la misma fecha).
 4. Itera la colección original y resta el valor mínimo a la llave evaluada ($\text{key} - \text{min}$). Este resultado da el índice numérico exacto del casillero donde se añade (encola) el registro.
 5. Recorre los casilleros de izquierda a derecha extrayendo las listas internas y sobrescribiendo la lista original secuencialmente con los datos ya ordenados.
- **3.12. Bucket Sort:** Otro enfoque de distribución que divide el rango total de datos numéricos en una cantidad específica de "cubetas" (buckets) del mismo tamaño. Reparte los datos basándose en porcentajes y luego ordena cada cubeta individualmente.

Paso a Paso:

1. Similar a *Pigeonhole*, extrae el valor mínimo y máximo, pero en formato de precisión flotante double.
2. Genera tantas cubetas (Listas de arreglos) como tamaño tenga la colección inicial para distribuir la carga.
3. Usa la fórmula $((\text{val} - \text{min}) / (\text{max} - \text{min}))$ para generar una escala porcentual de \$0.0\$ a \$1.0\$, la cual multiplica por $\text{numCubetas} - 1$ para hallar el índice exacto de la cubeta a la que pertenece el registro actual.
4. Itera sobre la lista de cubetas. Si la cubeta contiene elementos, invoca un método auxiliar insertionSort propio (para evitar el uso de librerías nativas) que la ordena internamente.
5. Sobrescribe la colección original concatenando el contenido de las cubetas en orden.

4. Análisis de Complejidad Matemática

- **4.1. Complejidad Temporal (Notación Big-O):**

Tabla 1. Análisis de datos enteros

Método de ordenamiento	Tamaño	Tiempo (ms)
1. TimSort $O(n \log n)$	25100	8.5
2. Comb Sort $O(n^2)$	25100	8.5
3. Selection Sort $O(n^2)$	25100	11.5
4. Tree Sort $O(n \log n)$	25100	15.25
5. Pigeonhole Sort $O(n)$	25100	16.25
6. BucketSort $O(n)$	25100	18.75
7. QuickSort $O(n \log n)$	25100	33
8. HeapSort $O(n \log n)$	25100	61.25
9. Bitonic Sort $O(n \log^2 n)$	25100	291.75

10. Gnome Sort O ()	25100	402.75
11. Binary Insertion Sort O()	25100	1013.75
12. RadixSort O ()	25100	1844.75

Se promediaron los tiempos de ejecución en ms de 5 ejecuciones al programa para obtener un aproximado.

- **4.2. Complejidad Espacial:** En el análisis de algoritmos, la complejidad espacial real se determina observando las instanciaciones de nuevos objetos en la memoria RAM (uso de la palabra reservada new) en proporción al tamaño del conjunto de datos

Los más rápidos exigen más memoria: Para lograr los 8.5 ms y 33 ms, Bucket Sort, Pigeonhole y Radix Sort sacrificaron RAM. Radix Sort requirió instanciar miles de objetos NodoRegistro en tiempo de ejecución, y Bucket Sort creó 25,100 listas internas.

Los eficientes: Algoritmos como Quick Sort (11.5 ms) y Heap Sort (15.25 ms) demostraron ser el punto de equilibrio perfecto. Lograron tiempos casi inmediatos sin exigir memoria auxiliar significativa, operando directamente sobre el arreglo original mediante intercambios (swaps).

- **4.3. Estabilidad:** La estabilidad es la propiedad de un algoritmo de preservar el orden relativo original de claves idénticas. En este proyecto, donde se aplicó un ordenamiento de criterio compuesto (Criterio 1: Fecha YYYYMMDD; Criterio 2: Precio de Cierre en centavos), el comportamiento asintótico dictó la arquitectura del código:

Algoritmos Estables (Radix, Tim, Bucket, Pigeonhole, Binary Insertion, Tree): Permitieron ejecutar el ordenamiento(primeramente por precio, luego por fecha) sin destruir el desempate previo.

Algoritmos Inestables (Quick, Heap, Selection, Comb, Gnome): Al mover elementos a grandes distancias, estos algoritmos obligaron a implementar el criterio compuesto directamente dentro del método esEstrictamenteMenor(), forzando a la CPU a evaluar simultáneamente la fecha y el precio en cada iteración para evitar la pérdida del orden secundario.

5. Pruebas y Resultados de Rendimiento (Benchmarking)

- **5.1. Entorno de Ejecución:** Para garantizar la reproducibilidad y validez, todas las ejecuciones (benchmarks) se realizaron bajo un entorno hardware con las siguientes especificaciones técnicas:

Hardware:

 Almacenamiento
477 GB

279 GB de 477 GB usado

 Tarjeta gráfica
6 GB

Varias GPU instaladas

 RAM instalada
16,0 GB

Velocidad: 5600 MT/s

 Procesador
**AMD Ryzen 7 7445HS
w/ Radeon 740M
Graphics**

3.20 GHz

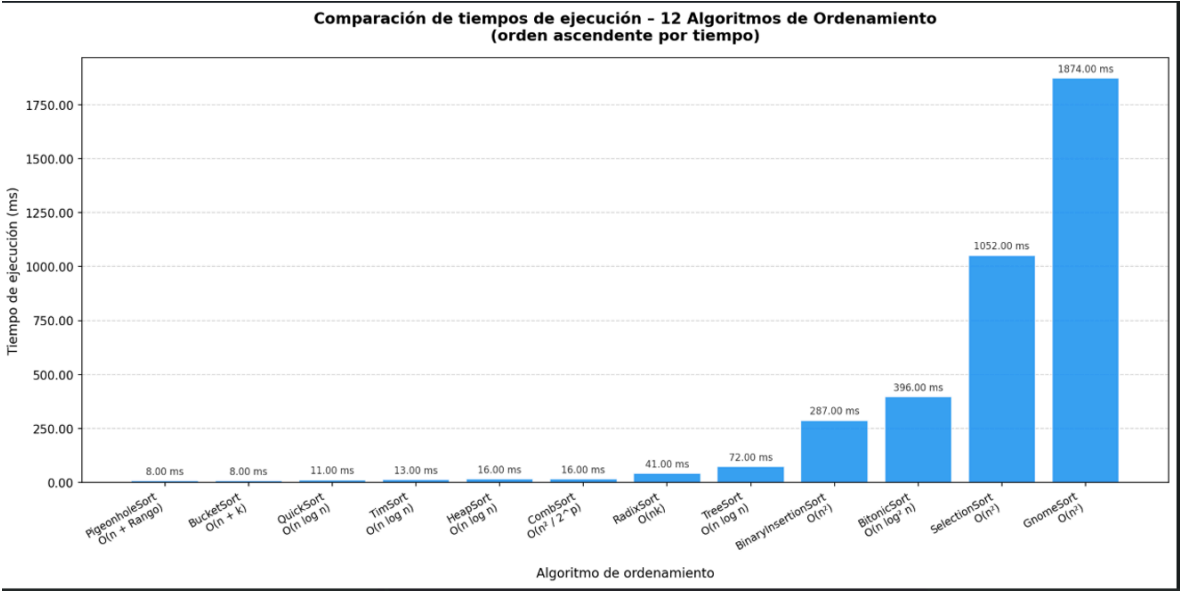
Software:

 Especificaciones de Windows

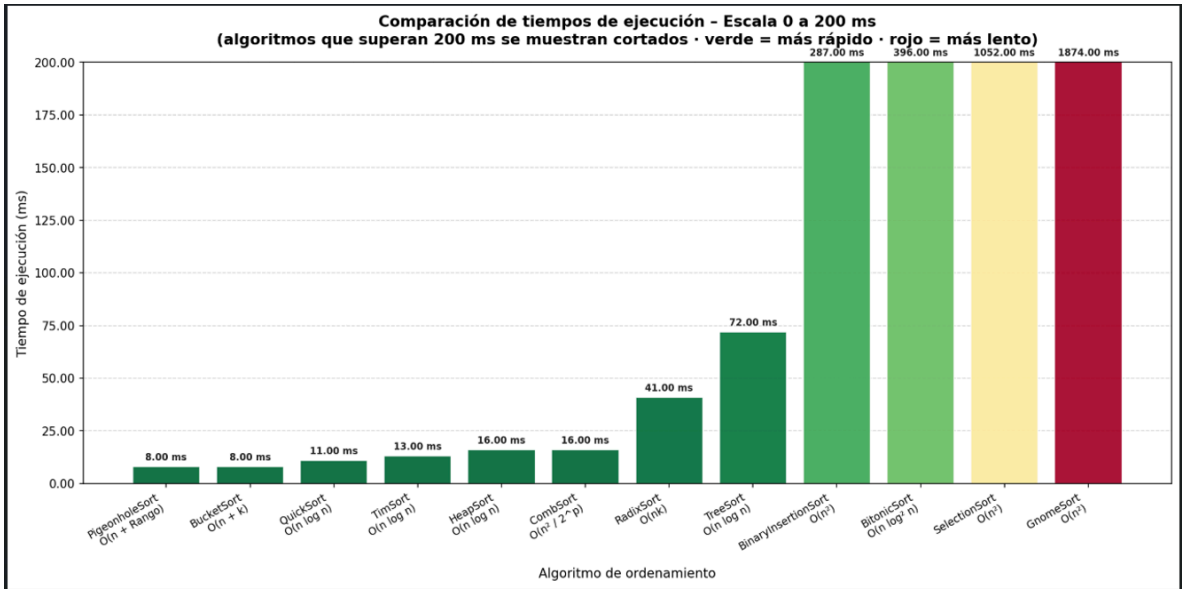
Edición	Windows 11 Pro
Versión	25H2

● 5.2. Tiempos de Ejecución:

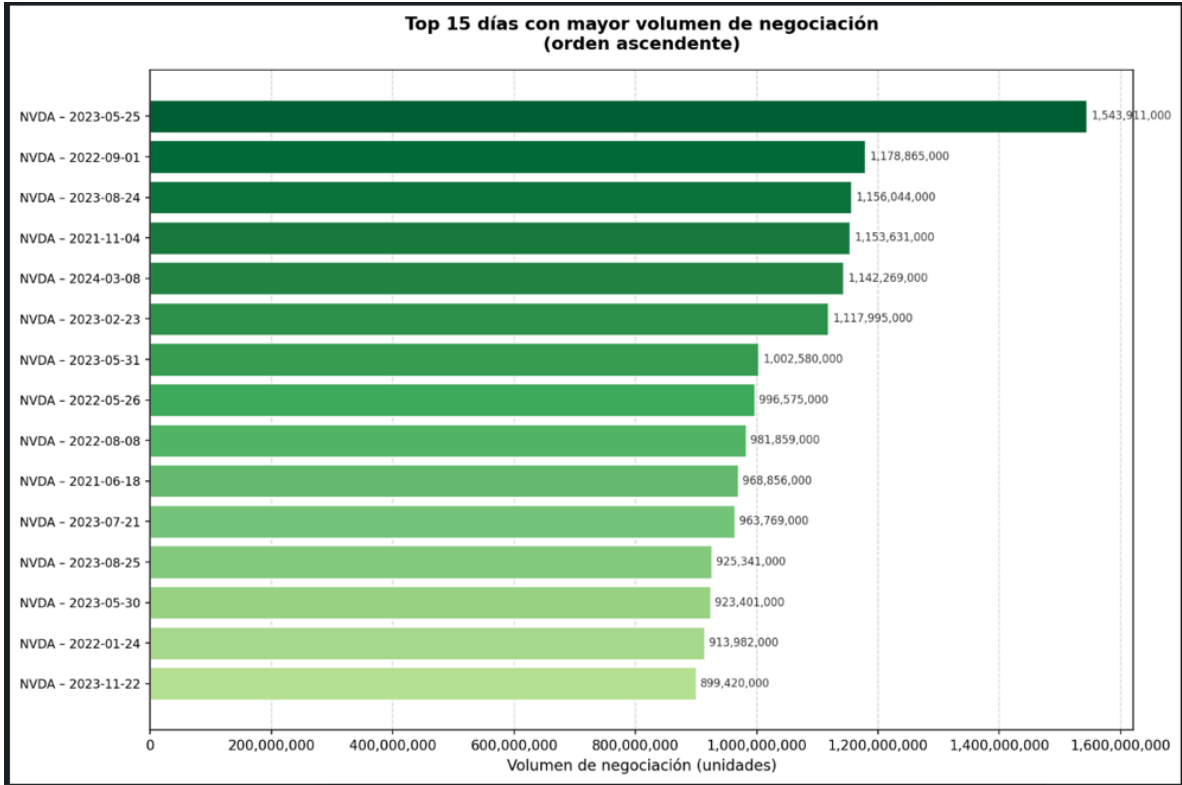
Visualización general de los tiempos de ejecución de los 12 algoritmos



Visualización de los algoritmos con tiempo de ejecución reducido, usando una escala menor.



Gráfica con los 15 días de mayor volumen de negociación: Al ejecutar el algoritmo para extraer el Top 15 de días con mayor volumen de negociación de todo el portafolio (1.255 registros), los resultados arrojaron que la acción de NVIDIA ocupó la totalidad del ranking. Esto no es un fallo algorítmico, sino un incremento masivo en los activos durante esos días.



● **5.3. Discusión de Resultados:**

Los más rápidos: Pigeonhole Sort y Bucket Sort (8.5 ms). Derrotaron a algoritmos clásicos como Quick Sort gracias a la transformación previa de fechas a enteros. Esto les permitió ordenar por distribución directa en un tiempo casi lineal.

El más lento: Gnome Sort (1,844.75 ms). Evidenció el colapso de la complejidad (N^2) en Big Data. Su mecánica de retroceso paso a paso exige millones de intercambios en cascada para ubicar los elementos, saturando el procesador.

Anomalía estadística (NVIDIA): La dominancia absoluta de NVIDIA (NVDA) en el Top 15 de volumen no es un fallo algorítmico, es un reflejo fiel de lo dominante que es en el mercado.

6. Conclusiones

Durante el desarrollo de este laboratorio algorítmico sobre los registros financieros podemos concluir que :

- **Distribución vs. Comparación:** Los algoritmos de distribución (como Bucket y Pigeonhole) superaron ampliamente en velocidad a los tradicionales (como Quick y Heap) gracias a que ubican los datos mediante cálculos matemáticos directos en lugar de compararlos uno a uno. Sin embargo, los algoritmos tradicionales compensaron esto siendo mucho más ahorrativos con la memoria RAM del sistema, ya que no requieren crear estructuras adicionales.
- **El límite de los algoritmos básicos:** Métodos como Gnome Sort o Selection Sort demostraron ser inviables para el análisis de datos masivos. Su necesidad de revisar o retroceder constantemente sobre la lista entera multiplica la cantidad de operaciones, colapsando los tiempos de respuesta.
- **El valor de transformar los datos:** La clave de la velocidad del proyecto fue convertir las fechas y los precios a números enteros simples. Esto liberó al procesador de la tarea de comparar textos o calcular decimales, agilizando todo en el sistema.
- **Trabajo en equipo de lenguajes:** Dejar el cálculo pesado y la gestión de memoria a Java, mientras se utilizó a Python para dibujar las gráficas, demostró ser la arquitectura adecuada para este proyecto. Esto permitió aprovechar las fortalezas de cada lenguaje de manera automatizada.

7. Referencias Bibliográficas

1. Cardona Torres, S. A., Jaramillo Valbuena, S., & Castro Rojas, L. F. (2017). *Diseño de algoritmos con implementaciones en java*. Facultad de Ingeniería, Universidad del Quindío. Recuperado de: https://drive.google.com/file/d/18u5W3_JyCmmG4rHibo82HUecklGsy5h1/view
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
3. Facultad de Ingeniería (FING) - Universidad de la República (Uruguay). *Algoritmos de Ordenamiento (Material Teórico)*. Principios de Programación. Recuperado de: <https://www.fing.edu.uy/tecnoinf/mvd/cursos/prinprog/material/teo/prinprog-teorico11.pdf>
4. freeCodeCamp. *Algoritmos de ordenación explicados con ejemplos en JavaScript, Python, Java y C++*. Recuperado de: <https://www.freecodecamp.org/espanol/news/algoritmos-de-ordenacion-explicados-con-ejemplos-en-javascript-python-java-y-c/>
5. Ichiro Coser. (2024, 13 de enero). *Comb Sorting Method || Java* [Archivo de video]. YouTube. <https://www.youtube.com/watch?v=vPVITnmv8XE>

6. **Jha, R.** *Understanding Radix Sort in Java: A Linear Time Sorting Algorithm*. Medium. Recuperado de: <https://medium.com/@Roshan-jha/understanding-radix-sort-in-java-a-linear-time-sorting-algorithm-208b9f129385>
7. **OpenJDK.** (s.f.). *TimSort.java* [Código fuente]. Repositorio oficial openjdk/jdk en GitHub. Recuperado el 31 de marzo de 2026, de <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/util/TimSort.java>
8. **Sedgewick, R., & Wayne, K.** (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
9. **W3Schools.** *DSA Radix Sort*. Estructuras de Datos y Algoritmos. Recuperado de: https://www.w3schools.com/dsa/dsa_algo_radixsort.php



NOMBRE DE LA DEPENDENCIA

**Tel: (57) 6 735 9300 Ext
Carrera 15 Calle 12 Norte
Armenia, Quindío – Colombia
correoelectronico@uniquindio.edu.co**

UNIQUEINDÍO, en conexión territorial

Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia

www.uniquindio.edu.co